MaxContext: Amortizing LLM Setwise Reranking with Whole-Pool Prompts

Anonymous ACL submission

Abstract

Setwise reranking with large language models (Zhuang et al., 2024b) prompts the model to select the most relevant document from a small candidate window and sorts these local decisions into a top-k ranking, extracting only one explicit extreme per LLM call — the identity of the best document — without explicitly using the worst document or the relative order of the rest. Asking *jointly* for the best and worst document in the same prompt (DualEnd) extracts two explicit extremes per call, but at the usual four-passage window it still requires hundreds of dependent LLM calls per query, leaving a large latency gap between lightweight setwise sorts and quality-oriented joint elicitation. We propose **MaxContext**, a family of zero-shot LLM rerankers that prompts repeatedly over the *whole live rerank pool* (up to 50 documents) instead of a small sliding window. The family has three variants that ask, on each call, for either joint best+worst (DUALEND), best-only (TOPDOWN), or worst-only (BOTTOMUP) selection from the live pool; MAXCONTEXT-DUALEND reranks a pool of size N in $\lfloor N/2 \rfloor$ dependent LLM calls, and the single-extreme variants in $N-2$ calls plus a deterministic two-document BM25 endgame. The three variants jointly test whether quality differences relative to small-window setwise arise from joint elicitation, from whole-pool selection itself, or from single-extreme selection at long context. We evaluate MaxContext on six Qwen3 / Qwen3.5 checkpoints (4B–27B) on TREC DL19 and DL20 against matched-`hits` TopDown-Heap, TopDown-Bubble, DualEnd-Cocktail, and DualEnd-Selection baselines, with explicit pool-size, passage-length, and input-order ablations. As a secondary fixed-configuration operating point that can scale beyond MaxContext's pool-size ceiling, we additionally evaluate a Top-m Setwise + DualEnd finalist cascade. We frame our claim as potential movement on the comparisons-axis and wall-clock-axis of the reranking Pareto frontier with transparent reporting of prompt, completion, and total tokens, parse-fallback rates, and order-stability diagnostics, so that any quality difference can be audited against its true compute cost rather than an isolated call-count headline.

1 Introduction

Large language models (LLMs) are now strong zero-shot rerankers in information retrieval (IR) pipelines. The dominant paradigms differ in how they elicit relevance judgements from the model: pointwise scoring evaluates one query-document pair at a time (Nogueira et al., 2020), pairwise prompting compares two documents per call (Qin et al., 2024), listwise prompting feeds an entire candidate list and asks for a permutation (Sun et al., 2023; Ma et al., 2023), and *setwise* prompting (Zhuang et al., 2024b) feeds a small candidate window of $c+1$ passages and asks the model to pick the most relevant one, embedding that comparator inside a classical sort (heapsort, bubblesort) to produce a top-k ranking. Setwise reranking has become a standard zero-shot baseline because it sits on the effectiveness–efficiency Pareto frontier between pointwise and listwise, while keeping prompts short enough to fit comfortably in any modern LLM's context.

Yet the setwise comparator is informationally thin. Each LLM call reads $c+1$ passages, internally evaluates all of them, and emits a single label. The algorithm uses only that one explicit extreme — the identity of the best document — and does not explicitly use the rest of the model's implicit comparison evidence. Two recent directions try to use each setwise call more effectively. Joint elicitation asks for both the best and worst document in one prompt (DUALEND), recovering two explicit extremes per call. Tournament prompting (Chen et al., 2025b) keeps the best-document comparator but embeds it in a bracket, so advancement, elimi-

nation, and accumulated points turn local winners into broader ranking evidence. Both directions improve the information-per-call envelope, but both remain anchored to bounded local groups rather than repeatedly prompting over the whole rerank pool.

The mismatch. Modern instruction-tuned LLMs comfortably handle contexts of 32K–256K tokens. The Qwen3 family used in this paper, for instance, ships with 32K native context for the 4B/8B/14B variants and 256K for the Qwen3.5 9B model. At a passage length of 512 tokens, a 50-document rerank pool plus query, scaffolding, and decoding budget consumes well under 30K tokens — it fits in one prompt. The setwise sliding window of size $c{+}1{=}4$ is therefore an artefact of an earlier compute regime: it forces hundreds of sequential LLM calls per query whose only role is to compose enough small local decisions into a top-k ranking, even though the entire rerank pool would fit in a single context window. In our completed baseline runs on TREC DL19/20, DUALEND-COCKTAIL averages 546 comparisons and roughly $8.9\times$ the wall-clock of a TopDown-Heap baseline, for a mean NDCG@10 gain of only $+0.0065$ over the cheapest setwise sort. The Pareto frontier between TopDown-Bubble (300 comparisons, 110.9 s, NDCG@10 0.6897) and DualEnd-Cocktail (546 comparisons, 212.6 s, NDCG@10 0.6962) is *empty*: none of our completed setwise / DualEnd baselines delivers DualEnd-class quality at TopDown-class cost.

This paper. We target this gap by abandoning the small-window assumption altogether. We introduce **MaxContext**, a family of zero-shot LLM rerankers that prompts the model over the *entire live rerank pool* on every call. Given a starting pool of N documents (we evaluate $N \in \{10, 20, 30, 40, 50\}$, all bounded by 50), MaxContext repeatedly sends the live pool to the LLM, removes the document(s) the model points to, and continues until the pool is exhausted. Three variants probe different hypotheses about *which* signal the long-context model should produce on each call:

- MAXCONTEXT-DUALEND asks for both the best and worst document in the live pool; the pool shrinks by two documents per call. Total cost: $\lfloor N/2 \rfloor$ LLM calls per query (e.g. 25 at $N{=}50$).
- MAXCONTEXT-TOPDOWN asks for only the best document; the pool shrinks by one. A two-document deterministic BM25 endgame resolves the last pair. Total cost: $N{-}2$ calls plus a constant-time tiebreaker.
- MAXCONTEXT-BOTTOMUP asks for only the worst document; the pool shrinks by one with the same BM25 endgame. Total cost matches TopDown but on the opposite end of the ranking.

Each call places one or two documents at globally observed rank positions, so the algorithm produces a permutation of the input pool with no early termination, no auto-batching, and no auto-truncation. We replace the legacy letter labels (`A`–`W`) with numeric labels 1–N to support $N > 23$, harden the parser against silent fallbacks at $N{=}50$, and introduce strict invariants (`pool_size = hits = ranker.k`, abort-on-bad-parse, no truncation) so that whole-pool prompts cannot be silently clipped or misinterpreted.

Why three variants. MaxContext is a method family rather than a single method: it implements a controlled comparison of comparator output types under a fixed whole-pool prompt structure. Three predeclared hypotheses are testable simultaneously:

1. **Joint-elicitation hypothesis.** Whole-pool quality comes from asking for two extremes per call. Predicts DUALEND $\succ$ TOPDOWN $\approx$ BOTTOMUP.
2. **Whole-pool-selection hypothesis.** Whole-pool quality comes from letting the model see all candidates simultaneously, regardless of how many outputs it emits. Predicts DUALEND $\approx$ TOPDOWN $\approx$ BOTTOMUP.
3. **Single-extreme hypothesis.** Joint elicitation does not scale to long context and degrades DUALEND relative to selection-only variants. Predicts one or both single-extreme variants outperform DUALEND.

Reporting all three variants under a common harness lets the empirical outcome itself decide which mechanism is responsible for any frontier movement, rather than committing in advance to one explanation.

Complementary operating point: Top-m + DualEnd cascade. MaxContext's hard ceiling at pool size ≤ 50 is set by the smallest context window in our model lineup. To probe the same Pareto-frontier region *without* a hard pool-size cap, we additionally evaluate a Top-m Setwise + DualEnd finalist cascade. The cascade asks the LLM to pick the top-m documents from each setwise group, accumulates votes across r stratified groupings, and then refines the surviving finalist pool with MaxContext-DualEnd. The cascade can in principle scale to larger initial pools at the cost of weaker recall guarantees, and so serves as a different operating point on the same comparisons/wall-clock frontier rather than as a competing main method.

Reporting discipline. A long-context whole-pool prompt is *not* a free lunch. It uses substantially more prompt tokens per call than a 4-passage setwise prompt; the total number of dependent LLM calls can drop by an order of magnitude (e.g., for MaxContext-DualEnd relative to small-window DualEnd-Cocktail), but per-call token cost rises in tandem. We therefore commit, throughout the paper, to reporting all five cost axes recommended by the recent reranker efficiency literature (Peng et al., 2025): total dependent LLM calls ("comparisons"), prompt tokens, completion tokens, total tokens, and end-to-end wall-clock latency. Our efficiency claim is framed strictly as *movement on the comparisons-axis and wall-clock-axis*, not as raw token efficiency. A reader interested in pure token cost will find TD-HEAP remains the cheapest option in our experiments; MaxContext is designed for readers interested in sequential-call count or end-to-end latency at competitive NDCG@10 quality.

Contributions.

1. **Method.** We introduce MaxContext, a zero-shot setwise LLM reranking family that prompts the model over the entire live rerank pool on every call. We give three variants spanning a controlled comparator-output ablation (joint best+worst, best-only, worst-only) and present the algorithmic, prompt-engineering, and parser-hardening details required to make whole-pool prompts robust at N=50.

2. **Pareto-frontier characterization.** We empirically characterize the previously empty region between TopDown-Bubble and DualEnd-Cocktail on the comparisons-axis and wall-clock-axis frontiers, with matched-`hits` baselines at three pool sizes and full transparent cost reporting.

3. **Mechanism diagnosis.** The three-variant ablation, combined with pool-size sweeps and an order-robustness pilot, lets us diagnose *whether* any quality difference comes from joint elicitation, whole-pool selection itself, or single-extreme selection at long context — a question that no existing setwise study can answer because none reaches large enough pools to expose the difference.

4. **Complementary operating point.** We additionally evaluate a Top-m Setwise + DualEnd cascade as a second operating point on the same frontier, providing a method that can in principle scale beyond the 50- document pool ceiling.

5. **Order-robustness and position-bias-at-scale diagnostics.** We predeclare a controlled-ordering pilot at N=50 and report selected-position frequencies under a numeric label scheme 1–N, separately from the w=4 letter-scheme position-bias evidence in prior work, so that any extension or contradiction of the small-window position-bias literature is reported as an empirical finding rather than presupposed.

The paper is organized as follows. Section 2 situates MaxContext among setwise extensions, tournament-style rerankers, and the long-context-attention literature. Section 3 formalises MaxContext and the Top-m+DualEnd cascade. Section 4 describes the staged evaluation matrix; Sections 5 and 6 report the matched-hits Pareto frontier, the mechanism-ablation, and the order-robustness diagnostics. Section 7 concludes.

2 Related Work

We organize related work along three axes: (§2.1) the LLM zero-shot reranking paradigms, with emphasis on setwise; (§2.2) recent setwise / tournament extensions that share the "more information per LLM call" goal but stay anchored to bounded local groups, including **TourRank**; and (§2.3) long-context attention and position-bias work that constrains what is realistic at N=50.

2.1 LLM Zero-Shot Reranking Paradigms

LLM-based zero-shot reranking has converged on four prompting paradigms. **Pointwise** methods score each query-document pair independently (Nogueira et al., 2020; Pradeep et al., 2021); they are fast and easy to parallelize, but ignore inter-document context. **Pairwise** methods compare two documents per call (Qin et al., 2024); they extract the strongest local signal but pay $O(n^2)$ comparisons in the all-pairs variant, mitigated by sorting-based heapsort and bubblesort wrappers. **Listwise** methods (Sun et al., 2023; Ma et al., 2023; Zhang et al., 2025; Ren et al., 2025) feed a sliding window of candidates and ask the LLM to emit a permutation; they are strong at moderate cost but suffer from output parsing fragility, window-boundary effects, and well-documented position bias (Liu et al., 2024). **Setwise** methods (Zhuang et al., 2024b) sit between pairwise and listwise: each LLM call processes a small set of $c{+}1$ passages and emits a single label identifying the most relevant one; the comparator is then embedded in heapsort or bubblesort to produce a top-k ranking. We adopt the setwise paradigm and the public reference implementation (Zhuang et al., 2024a) as our baseline lineage.

The original setwise formulation (Zhuang et al., 2024b) fixes $c{+}1{=}4$, uses single-letter labels (`A`, `B`, `C`, `D`), and supports both *generation* scoring (decode the winning label) and *likelihood* scoring (compare logits of candidate label tokens, on encoder-decoder backbones). MaxContext extends this lineage in three respects: (i) the prompt feeds the entire live rerank pool rather than a small window; (ii) labels are numeric 1–N to support $N \geq 23$ under strict parser invariants; and (iii) the comparator can emit either one (TopDown, BottomUp) or two (DualEnd) labels per call. We use generation scoring throughout because joint best-worst elicitation requires the model to emit two distinct labels in a single response, which the likelihood scoring path cannot represent without collapsing to a best-only proxy.

2.2 Setwise and Tournament Extensions

Several recent papers share MaxContext's high-level goal of extracting more ranking evidence per LLM call, but each is anchored to bounded local groups of $c{+}1{\leq}20$.

Joint best-worst elicitation. Building on the public setwise reference implementation (Zhuang et al., 2024a), we implement a DualEnd comparator that asks the LLM to identify the most *and* least relevant document in a single prompt, doubling the explicit information per call from one label to a partition of the $c{+}1$ candidates into two extremes plus the rest. The dual-output comparator naturally pairs with cocktail-shaker sort or double-ended selection sort. In our completed baseline experiments on TREC DL19/20 across nine LLMs, DualEnd is the strongest small-window family but pays $5.6{-}8.9\times$ the wall-clock of TopDown-Heap. We treat small-window DualEnd as a *baseline* for this paper rather than a contribution: MaxContext-DualEnd inherits the joint best-worst comparator and applies it to the entire live pool instead of a $c{+}1{=}4$ window.

Tournament prompting: TourRank. TourRank (Chen et al., 2025b) runs setwise comparisons inside a multi-round tournament bracket. Documents are seeded into groups using BM25 ranks, each round uses an LLM to advance a fraction of each group's candidates to the next round, and a final ranking is produced by accumulated points across r tournaments. The information advantage over plain setwise heapsort is structural: a single comparator output provides broader ranking evidence through advancement, elimination, and accumulated points across rounds, and grouping seeded by initial rank reduces the chance that all strong candidates are concentrated in one group. TourRank is highly parallelizable — groups within a round are independent, and tournaments are independent of each other — and reports state-of-the-art quality on TREC DL and BEIR at moderate call budgets. Recent work (BlitzRank Authors, 2026) generalizes the tournament intuition into principled tournament *graphs* that explicitly model all preferences extractable from a k-way comparison, rather than only winner identity. MaxContext is complementary to both threads: it does not change the bracket / graph topology, but instead changes the *size* of the live group inside which a setwise comparator operates, pushing it from $c{+}1{\leq}20$ all the way to $N{=}50$.

Other setwise efficiency improvements. **Setwise Insertion** (Podolak et al., 2025) warm-starts the sort using the BM25 ranking as a prior, reducing query latency by $\sim$ 31% and the number of inferences by $\sim$ 23% without changing the comparator. This is orthogonal to MaxContext: an insertion-style warm start could be applied

within MaxContext's pool-shrinking loop, though we leave that to future work. **Rank-R1** (Zhuang et al., 2025) extends the setwise comparator with a reasoning chain learned via reinforcement learning and uses 20 candidates per call — a known-feasible precedent for moderately large setwise windows on Qwen-class models. Rank-R1 is complementary: it changes the *quality* of each comparison via training, while MaxContext changes the *scale* of each comparison via prompt-engineering at zero-shot. Combining a reasoning-trained comparator with whole-pool prompting is again future work. **REALM** (Wang et al., 2025) models per-document relevance as Gaussian distributions and uses recursive Bayesian updates to reduce LLM queries by 23–84%. MaxContext takes the opposite extreme of REALM's design space: rather than adapting the sort to the comparator's noise, it makes the comparator's window so large that the sort itself collapses to a few rounds of "pick-and-place from the live pool."

Other paradigm-level alternatives. **Attention-based reranking** (Chen et al., 2025a) skips generation entirely and uses attention scores from the LLM's forward pass as relevance weights, giving order-of-magnitude latency reductions at some quality cost; it is a different paradigm from setwise prompting. **BlockRank** (Gupta et al., 2025) exploits inter-document block sparsity in attention to scale in-context ranking to long candidate lists within a single forward pass, emphasising raw throughput. MaxContext differs from both in that it remains within the setwise prompting paradigm and pays the cost of repeated whole-pool generation calls in exchange for the ability to elicit best, worst, or both extremes from a long-context comparator. Reasoning-enhanced rerankers such as **Rank1** (Rank1 Authors, 2025) use test-time compute to improve a pointwise comparator; the techniques are stackable but address a different axis (per-call quality, not per-query call count).

2.3 Long-Context Attention, Position Bias, and Order Robustness

Whole-pool prompting at $N{=}50$ pushes a $\sim$ 25K-token prompt through the LLM, which exposes MaxContext to two well-studied risks.

Lost-in-the-middle. Liu et al. (2024) show that LLMs disproportionately attend to information at the beginning and end of long contexts, with a documented "U-shaped" performance curve. Hutter et al. (2025) extend the phenomenon to retrieval-augmented generation, finding that bias depends on document type and model architecture. Tang et al. (2024) propose *permutation self-consistency* for listwise reranking: shuffle the input list multiple times, run the LLM on each permutation, and aggregate the rankings. This mitigates position bias at the cost of multiplying inference. MaxContext does *not* apply permutation self-consistency by default because doing so would multiply the call count we are trying to reduce; instead, we treat order robustness as a launch gate, running a controlled-ordering pilot (BM25-forward, BM25-inverse, random) at $N{=}50$ and reporting the maximum pairwise NDCG@10 difference as evidence for or against order stability at scale.

LLM comparator inconsistency. Zeng et al. (2024) document two intrinsic inconsistencies in LLM-based comparison: order inconsistency (swapping inputs gives different outputs) and transitivity inconsistency (preference cycles). Sorting-based setwise approaches assume both. MaxContext does not eliminate these inconsistencies, but it reduces *exposure* to them: with only $\lfloor N/2 \rfloor$ or $N{+}2$ comparator calls per query, fewer opportunities for inconsistent outputs accumulate compared to small-window sorts that make several hundred calls. Order sensitivity is diagnosed by the controlled-ordering pilot in §4; format reliability is diagnosed separately by parse-fallback rates at $N{=}50$.

Sorting with noisy comparators and efficiency reporting. Sato (2026) surveys sorting algorithms specifically for LLM-based comparators, noting that classical complexity bounds may not be optimal when comparisons are expensive, batchable, and noisy. Recent work also argues that LLM-reranker efficiency cannot be summarized by call count alone. Peng et al. (2025) introduce hardware-independent ranking-per-PetaFLOP and queries-per-PetaFLOP metrics, recognising that a single LLM call with 25K prompt tokens is fundamentally different from 25 small-window calls with 1K tokens each. We adopt the spirit of this recommendation: throughout the experimental sections we report all five cost axes (LLM calls, prompt tokens, completion tokens, total tokens, wall-clock) on every method, and frame our claim narrowly as potential Pareto-frontier movement on the comparisons-axis and wall-clock-axis rather than on per-token efficiency.

2.4 Positioning of MaxContext

Within this landscape, MaxContext is the first setwise reranker we are aware of that prompts over the entire live rerank pool on every call (up to N=50 documents) while retaining a controlled comparison of comparator output type via the three-variant ablation. It is complementary to TourRank-style tournament structure (which changes *how* comparator outputs are aggregated), to Setwise Insertion (which changes *where* the sort starts), and to Rank-R1 (which changes *what* the comparator internally computes). Unlike permutation self-consistency methods, MaxContext treats order robustness as an empirical constraint rather than as a mitigation strategy: we evaluate controlled input orders rather than multiplying inference by default. The natural follow-up question — whether stacking MaxContext with a tournament structure or a reasoning-trained comparator yields further improvements — is left to future work.

3 Methodology

[To be drafted in a subsequent revision once Phase 4 results land. This section will formalize the MaxContext family — whole-pool single-prompt joint best-worst elicitation, plus its single-extreme variants — alongside a complementary Top-m Setwise+Dualend operating point that targets the same empty Pareto-frontier region from a different mechanistic angle.]

4 Experimental Setup

We organize the evaluation around three questions, each addressed by a predeclared sub-study: **(Q1)** does whole-pool prompting move the Pareto frontier on the comparisons-axis and wall-clock-axis at matched-`hits` cost? **(Q2)** which mechanism — joint elicitation, whole-pool selection, or single-extreme selection — is responsible for any movement? **(Q3)** how does whole-pool prompting behave under input-order perturbations and varying passage lengths? The matrix below is staged so that order-stability and per-query parse reliability are validated before the bulk of compute is committed.

4.1 Datasets and First-Stage Retrieval

We evaluate on the standard zero-shot LLM reranking benchmarks: **TREC Deep Learning 2019** (Craswell et al., 2020) and **TREC Deep Learning 2020** (Craswell et al., 2021), both built on the MS MARCO passage collection. DL19 contains 43 judged queries with graded relevance, DL20 contains 54. First-stage retrieval is BM25 (Robertson and Zaragoza, 2009) as implemented in Pyserini (Lin et al., 2021), retrieving the top-N candidates per query for $N \in \{10, 20, 30, 40, 50\}$. Matched-`hits` discipline (see §4.4) requires that every reranking method is evaluated against a baseline that re-ranked the *same* BM25 top-N list, since archived runs at `hits`=100 cannot be subsetted to `hits`=50 without changing the candidate pool that the reranker ever sees.[1] All experiments report nDCG@10 (primary). We also log nDCG@100, MAP@10, MAP@100, and Recall@1000 against the official qrels via `trec_eval`; secondary metrics are reported in the appendix unless space permits inclusion in the main tables.

4.2 Models

We instantiate MaxContext on six instruction-tuned Qwen checkpoints (Qwen Team, 2025): **Qwen3-4B**, **Qwen3-8B**, **Qwen3-14B**, **Qwen3.5-4B**, **Qwen3.5-9B**, and **Qwen3.5-27B**. We restrict to Qwen because only the generation-scoring code path on Qwen3 / Qwen3.5 in our implementation exercises true joint best-worst elicitation as a single decoding step; encoder-decoder backbones such as Flan-T5 (Chung et al., 2024) and the likelihood-scoring path collapse joint elicitation to a best-only proxy via argmax / argmin over independent label likelihoods. **All MaxContext claims in this paper are therefore scoped to Qwen-generation models; we do not claim model-family generality.** Llama- and Mistral-based extension is conditional on family-specific sanity-smoke validation (parse stability at N=50, order robustness, refusal-pattern coverage) and is out of scope for this submission.

Two contextual notes apply across the model lineup. **Native context length:** Qwen3-4B,8B,14B ship with 32,768-token native context; Qwen3.5-9B and the larger Qwen3.5 checkpoints ship with 262,144-token native context. At passage length 512, a 50-document pool plus query, prompt scaffolding, and decoding budget consumes well under 30K tokens, so no model in our lineup is ever close to its native limit. **Thinking mode disabled:** Qwen3 / Qwen3.5 generation runs use `enable_thinking=False`, capping the decoding budget at ∼128 output tokens to discourage long

[1] The reference run-script reads only the top-`hits` BM25 candidates before any LLM call, so a `hits`=50 rerun is a genuinely different experiment from a `hits`=100 run with post-hoc top-50 truncation.

chain-of-thought and keep the parse target to a structured `Best: <int>, Worst: <int>` (or single-int) string.

4.3 MaxContext Configuration

All MaxContext runs use a fixed harness with hard invariants. Each ranker asserts at construction time that `pool_size` = `hits` = `ranker_k`, that `num_permutation` = 1, that the model family is in the Qwen3 / Qwen3.5 allowlist, and that the runtime-rendered prompt fits inside the model's context budget with 128 tokens reserved for decoding. Truncation is forbidden: any prompt that would exceed the budget aborts the run with a diagnostic. Document labels are numeric ($1..N$); the parser tightens the upstream regex to validate that extracted integers lie in $[1, N]$ and to reject duplicate or out-of-range labels with no silent default. Out-of-range labels and duplicate best/worst labels trigger an immediate run-abort rather than a fallback ranking. For the single-extreme variants, an n_{docs}=2 deterministic BM25 endgame resolves the last pair: MAXCONTEXT-TOPDOWN keeps the higher-BM25 document at the next top rank (so the BM25 endgame decides ranks $N-1$ vs. N, the tail of the ranking), while MAXCONTEXT-BOTTOMUP keeps the lower-BM25 document at the next bottom rank (so the BM25 endgame decides ranks 1 vs. 2, the head of the ranking). Because nDCG@10 is sensitive to head-of-ranking decisions, we report TOPDOWN and BOTTOMUP on the Pareto plot *separately* and decline any causal claim about whether the single-extreme bypass position causes the observed quality gap.

4.4 Baselines

Matched-`hits` setwise / DualEnd lineage. For every `hits` = pool-size value $N \in \{10, 30, 50\}$ (the cheapest, mid-range, and deepest anchors of our pool sweep), we re-rerank the same BM25 top-N list with four predeclared baselines: **TopDown-Heap (TD-Heap)** and **TopDown-Bubble (TD-Bubble)** (both upstream setwise sorts at `num_child`=3); **DualEnd-Cocktail (DE-Cocktail)** (cocktail-shaker sort over a joint best-worst comparator); and **DualEnd-Selection (DE-Selection)** (double-ended selection sort over the same comparator). All four use the public setwise reference codebase (Zhuang et al., 2024a), our DualEnd extension built on top, and the same six Qwen models, two datasets, and passage length 512 as the MaxContext primary runs. If Phase 2's order-stability gate (see below) fails at N=50 and forces the production matrix to a smaller pool size, the matched-`hits` baseline grid is extended to include the chosen smaller N (N=20 by default) so that the non-inferiority rule remains evaluated against same-`hits` baselines.

The headline rule for **(Q1)** is non-inferiority: at matched `hits` N, MAXCONTEXT-DUALEND must be no worse than the *best of* {DE-Cocktail, DE-Selection, TD-Bubble} on nDCG@10 with a predeclared margin of $\Delta \geq -0.003$ (point estimate) and a bootstrap-CI lower bound of ≥ -0.005, while reducing the average per-query LLM call count. We use the best-of rule rather than a single baseline because DE-Selection holds the only Bonferroni-significant DualEnd win in our completed small-window experiments; comparing only against DE-Cocktail would effectively cherry-pick an easier baseline. If a single-extreme variant rather than DualEnd dominates the Phase 4 frontier, the same non-inferiority rule is reported for that variant against *same-direction* matched-`hits` baselines (TD-Heap or TD-Bubble for MAXCONTEXT-TOPDOWN; the BU-Heap upstream baseline for MAXCONTEXT-BOTTOMUP), so the headline framing scales with whichever mechanism is empirically responsible for the frontier movement.

Why TourRank is not a primary baseline. Our primary comparisons are within the matched-`hits` setwise / DualEnd family because the main claim concerns whole-pool prompting under controlled pool depth. TourRank (Chen et al., 2025b) varies different budget knobs — number of tournaments, group size, advancement fraction, and parallel tournament schedule — so a fair head-to-head requires a separate matched-budget sweep. We report TourRank as related work (§2.2) and leave a full MaxContext–TourRank comparison to future work. **TourRank is therefore not part of the non-inferiority rule above; the matched-`hits` non-inferiority claim is internal to the setwise / DualEnd family.**

Top-m + DualEnd cascade. The complementary operating point is evaluated under the same Qwen lineup, datasets, and passage length. The cascade starts from the BM25 top-100 candidates — a deeper pool than the matched-`hits` setwise baselines, since the cascade's role is to probe whether prefiltering with Top-m selection lets us refine over a larger initial pool than MaxContext alone can

fit in context. Hyperparameters are predeclared as $G \in \{10, 20\}$ (setwise group size at the Top-m stage), m=5, $r \in \{1, 3\}$ (repeated stratified groupings), and a finalist pool capped at $|F|$=30 that is refined directly with MaxContext-DualEnd ($\lfloor |F|/2 \rfloor$ dependent calls; no sharding required at $|F| \leq 50$). The finalist scoring rule is predeclared:

$$\text{score}(d) = \text{vote}^{+}(d) - \lambda\, \text{vote}^{-}(d) + \beta\, \text{prior}_{\text{BM25}}(d),$$

with λ=0.25, β=0.10, and deterministic tie-breaks (higher positive vote rate first, then better BM25 rank, then stable document ID). *Per-query normalization:* $\text{vote}^{+}(d)$, $\text{vote}^{-}(d)$, and $\text{prior}_{\text{BM25}}(d)$ are each min–max scaled to $[0, 1]$ within the query before being combined, so the λ / β hyperparameters operate on a comparable scale across queries. Cascade hyperparameters are reported as a fixed sensitivity check rather than as a tuned competing method, and finalist-pool recall (the fraction of qrels-relevant documents retained in F) is reported separately as a diagnostic of how aggressively the Top-m stage prunes.

4.5 Cost Reporting and Statistics

Five cost axes. Following Peng et al. (2025), every reported method has all five axes recorded and figured: (i) average per-query *dependent* LLM calls, labelled "Avg comparisons" for parity with prior setwise tables; (ii) average per-query prompt tokens; (iii) average per-query completion tokens; (iv) average per-query total tokens; (v) end-to-end wall-clock latency, measured on identical NVIDIA H100 GPUs via Vast.ai. We also log per-call parse status (`full_parse`, `fallback_parse`, or `abort`) and report the *parse-fallback rate* as the fraction of LLM calls per query with a `fallback_parse`, averaged over the test set. `fallback_parse` indicates the parser used a secondary extractor (e.g., a numeric-only refusal regex) and recovered a valid label within $[1, N]$; an `abort` indicates an out-of-range label, duplicate best/worst labels, or refusal that the parser cannot recover from — aborts halt the entire run rather than substitute a fallback ranking. **We do not claim token efficiency:** on prompt-token and total-token axes, MaxContext is expected to be more expensive than small-window setwise; the only legitimate efficiency claims are on the comparisons-axis and wall-clock-axis.

Significance testing. Pairwise differences in per-query nDCG@10 are tested with paired approximate-randomization tests. Bonferroni correction is applied across the within-family comparison set (number of model-dataset configurations per family, typically 12 within Qwen). Bootstrap 95% confidence intervals on the per-query nDCG@10 difference are reported alongside point-estimate deltas, so non-inferiority claims combine point and interval evidence per §4.4.

4.6 Staged Matrix and Launch Gates

The full MaxContext production matrix runs on a shared SLURM cluster. Total sizing is approximately **420 runs** ($\sim$840 H100 hours of compute) across DualEnd, TopDown, and BottomUp pool sweeps plus the matched-`hits` baseline grid; the mandatory pre-production gate (Phases 2 and 3 below) adds $\sim$56 H100 hours, and the contingent smaller-pool fallback adds another $\sim$36 H100 hours. Detailed launchers and command lines are released with the code.

Phase 1 – Unit sanity. Single-GPU smoke test on Qwen3-4B, DL19, $N \in \{10, 20, 30, 40, 50\}$, $pl = 512$. Pass criteria: zero parse aborts, zero truncations, output is a permutation of the input pool. Phase 1 has already passed for all three MaxContext variants in our development runs.

Phase 2 – Order-robustness pilot (Q3 launch gate). On Qwen3-4B and Qwen3.5-9B, DL19 and DL20, at N=50 and pl=512, we run MaxContext-DualEnd with three input orderings: BM25-forward (as retrieved), BM25-inverse, and a fixed-seed random shuffle (seed $= 42$ for reproducibility). **Pass criterion (primary):** maximum pairwise nDCG@10 difference across the three orderings must be ≤ 0.01 for both models on both datasets. **Phase 2 also produces three secondary position-bias-at-scale diagnostics** that are reported regardless of pass / fail:

- *Selected-position concentration:* per-call frequency at which each in-prompt position $i \in [1, N]$ is selected as best (and as worst, under DualEnd). Concentration $> 30\%$ at any single position under forward ordering is flagged.
- *Selected-doc stability:* fraction of queries on which the same docid is chosen between forward and inverse orderings; $< 50\%$ is flagged as order-dependent.
- *Parse-fallback location distribution:* the in-prompt position(s) most often associated with

parse-fallback events, used to inform parser hardening before Phase 4.

Failing the primary gate at N=50 re-targets the production matrix to the contingent fallback at N=20 (where context-attention degradation is less likely); persistent failure at N=20 aborts the pivot. Position- bias-at-scale results are reported separately from the w=4 letter- scheme analyses in our prior work, since small-window evidence does not automatically extend to N=50.

Phase 3 – Matched-`hits` non-inferiority (Q1 launch gate). Four (`model`, `dataset`) pairs (Qwen3-8B and Qwen3.5-9B × DL19 and DL20) compare MAXCONTEXT-DUALEND at N=50 against {DE-Cocktail, DE-Selection, TD-Bubble} at matched `hits`=50 under the best-of non-inferiority rule of §4.4. The DE-Cocktail and DE-Selection runs are launched fresh for the four pairs; the TD-Bubble runs at `hits`=50 are part of the same Phase 4B matched-`hits` baseline grid (and so are not double-counted in the Phase 3 gate's compute budget unless the corresponding TD-Bubble run is not yet on disk). **Pass criterion:** non-inferiority on at least 3 of 4 pairs. Phase 3 is a *launch gate*, not a final generalization claim: a 4-pair test on the small TREC DL test sets is large enough to catch model-specific or dataset-specific quirks that would otherwise contaminate Phase 4, but the final evidence must come from the full Phase 4 6-model × 2-dataset production matrix with bootstrap CIs and paired tests across all 12 configurations.

Phase 4 – Production sweeps (Q1 + Q2). Conditional on Phases 2–3 passing. Three sweeps: (i) Study A — pool-size sweep $N \in \{10, 20, 30, 40, 50\}$ for MAXCONTEXT-DUALEND on six Qwen models × two datasets (60 runs); (ii) Phase 4D — the same sweep for MAXCONTEXT-TOPDOWN (60 runs); (iii) Phase 4E — for MAXCONTEXT-BOTTOMUP (60 runs); plus the Phase 4B matched-`hits` baseline grid at $N \in \{10, 30, 50\}$ (or $\{10, 20, 30, 50\}$ if Phase 2 forced the smaller-pool fallback) across four sort families (144 runs at the default grid). Phase 4 produces the headline Pareto plots and the three-variant mechanism ablation.

Phase 5 – Passage-length sensitivity (Q3). At the predeclared production pool size from Phase 4, the passage-length sweep $pl \in \{64, 128, 256, 512\}$ runs both MAXCONTEXT-DUALEND (treatment, 48 runs) and a DE-Selection-with- `num_child`=3 control arm at the same pl (48 runs). Confound rule: if shorter pl wins under MaxContext but not under the control, attribute to long-context attention degradation; if shorter pl wins under both, attribute to passage-length sensitivity in the underlying Qwen scoring; any other pattern triggers further investigation before any attribution.

4.7 Reproducibility

All MaxContext code, prompt templates, parser, and launcher scripts are released under the same open-source licence as the underlying setwise codebase. Each completed run emits a JSON Lines log containing per-query LLM-call traces (prompt, response, parse status, label scheme), the deterministic seeds used for input-ordering pilots, the SLURM job ID, and the model checkpoint hash, so that downstream analyses — in particular the position-bias-at-scale stratification — can be re-derived without re-running the LLM. Cluster wall-clock numbers reported in the paper are the median across three independent re-runs of a 3-query subset of the official qrels per (model, dataset, method) configuration to absorb shared- cluster GPU-utilization variance.

5 Results

[To be drafted once Phase 4 / Phase 5 of the experimental matrix lands. This section will report: (i) Pareto-frontier comparison of MaxContext-DualEnd / TopDown / BottomUp against best-of-baseline at matched `hits` *on the comparisons-axis, total-tokens-axis, and wall-clock-axis; (ii) pool-size saturation curves (Study A); (iii) passage-length sweep with DualEnd-nc3 control arm (Study B); (iv) order-robustness diagnostics from Study C; and (v) the Top-m+Dualend operating-point comparison.]*

6 Analysis

[To be drafted once experimental results are available. Planned content: position-bias-at-scale (numeric labels $1..N$ at $N = 50$, separated from the $w = 4$ letter-scheme position-bias log used by prior work in this project), parse-fallback rate distributions, long-context attention degradation diagnostics, and per-query help/hurt analysis.]

7 Conclusion

[To be drafted once results land. Will summarize: (i) MaxContext amortizes joint best-worst elicitation across far fewer sequential LLM calls by fitting the entire rerank pool into a single prompt; (ii) the empirical characterization of joint vs. single-extreme whole-pool selection at pool size ≤ 50; (iii) the resulting Pareto-frontier movement; and (iv) the implications for setwise LLM reranking design.]

Limitations

[To be drafted with empirical results. Will document:

- ***Model-family scope.*** *MaxContext is currently restricted to the Qwen-generation code path because only Qwen exercises true joint best-worst elicitation in our implementation; T5-encoder-decoder and likelihood-scoring paths collapse to a best-only proxy. Llama and Mistral expansion is gated on sanity-smoke validation.*
- ***Pool-size cap.*** *MaxContext is evaluated at pool size ≤ 50. Larger pools require either explicit context budgeting or a different mechanism such as the Top-m+Dualend cascade (see Methodology).*
- ***Statistical fragility on TREC DL.*** *TREC DL19/20 have 43 and 54 queries respectively, and bootstrap confidence intervals on* NDCG@*10 are wide; we therefore report Bonferroni-corrected p-values and bootstrap CIs honestly rather than relying on point-estimate margins.*
- ***Single-extreme BM25 endgame asymmetry.*** *The two-document deterministic endgame in MaxContext-TopDown resolves ranks $N-1$ vs. N (low-impact tail), while MaxContext-BottomUp's resolves ranks 1 vs. 2 (high-impact head); we report the two variants separately and refrain from treating their difference as a causal claim about NDCG@10.*

]

References

BlitzRank Authors. 2026. BlitzRank: Principled tournament graphs for zero-shot LLM reranking. *arXiv preprint arXiv:2602.05448*.

Shijie Chen, Bernal Jimenez Gutierrez, and Yu Su. 2025a. Attention in large language models yields efficient zero-shot re-rankers. In *Proceedings of the 13th International Conference on Learning Representations*.

Yiqun Chen, Qi Liu, Yi Zhang, Weiwei Sun, Xueguang Ma, Wei Yang, Daiting Shi, Jiaxin Mao, and Dawei Yin. 2025b. TourRank: Utilizing large language models for documents ranking with a tournament-inspired strategy. In *Proceedings of the ACM Web Conference 2025*.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, and 16 others. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53.

Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. 2021. Overview of the TREC 2020 deep learning track. In *Proceedings of the Twenty-Ninth Text REtrieval Conference (TREC 2020)*.

Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. 2020. Overview of the TREC 2019 deep learning track. In *Proceedings of the Twenty-Eighth Text REtrieval Conference (TREC 2019)*.

Nirmit Gupta, Cong You, Srinadh Bhojanapalli, Sanjiv Kumar, Inderjit Dhillon, and Felix Yu. 2025. Scalable in-context ranking with generative models. In *Advances in Neural Information Processing Systems*.

Jens Hutter, David Rau, Maarten Marx, and Jaap Kamps. 2025. Lost but not only in the middle: Positional bias in retrieval augmented generation. In *Proceedings of the 47th European Conference on Information Retrieval*, pages 247–261.

Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. 2021. Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2356–2362.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*.

Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. 2023. Zero-shot listwise document reranking with a large language model. *arXiv preprint arXiv:2305.02156*.

Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. 2020. Document ranking with a pre-trained sequence-to-sequence model. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.

Zhiyuan Peng, Ting-Ruen Wei, Tingyu Song, and Yi Fang. 2025. E2R-FLOPs: Hardware-independent efficiency metrics for LLM-based rerankers. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*.

Jakub Podolak, Leon Peric, Mina Janicijevic, and Roxana Petcu. 2025. Beyond reproducibility: Advancing zero-shot LLM reranking efficiency with setwise insertion. In *Proceedings of the 48th International ACM SIGIR Conference*.

Ronak Pradeep, Rodrigo Nogueira, and Jimmy Lin. 2021. The Expando-Mono-Duo design pattern for text ranking with pretrained sequence-to-sequence models. *arXiv preprint arXiv:2101.05667*.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Rank1 Authors. 2025. Rank1: Test-time compute for reranking in information retrieval. In *Conference on Language Modeling (COLM)*.

Ruiyang Ren, Yuhao Wang, Kun Zhou, Wayne Xin Zhao, Wenjie Wang, Jing Liu, Ji-Rong Wen, and Tat-Seng Chua. 2025. Self-calibrated listwise reranking with large language models. In *Proceedings of the ACM Web Conference 2025*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Sato. 2026. A survey of sorting algorithms with LLM-based comparators. *DPC Technical Report DPC-TR-2026-001*.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? Investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Raphael Tang, Xinyu Zhang, Xueguang Ma, Jimmy Lin, and Ferhan Ture. 2024. Found in the middle: Permutation self-consistency improves listwise ranking in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*.

Pinhuan Wang, Zhiqiu Xia, Chunhua Liao, Feiyi Wang, and Hang Liu. 2025. REALM: Recursive relevance modeling for LLM-based document re-ranking. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Yifan Zeng, Ojas Tendolkar, Roger Baartmans, Qingyun Wu, Lizhong Chen, and Huazheng Wang. 2024. LLM-RankFusion: Mitigating intrinsic inconsistency in LLM-based ranking. In *NeurIPS 2024 Workshop on Adaptive Foundation Models*.

Xinyu Zhang, Sebastian Hofstätter, Patrick Lewis, Raphael Tang, and Jimmy Lin. 2025. Rank-without-GPT: Building GPT-independent listwise rerankers on open-source large language models. In *Proceedings of the 47th European Conference on Information Retrieval*.

Shengyao Zhuang, Bevan Koopman, Guido Zuccon, and contributors. 2024a. ielab/llm-rankers: Reference implementation of LLM-based reranking. `https://github.com/ielab/llm-rankers`.

Shengyao Zhuang, Xueguang Ma, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2025. Rank-R1: Enhancing reasoning in LLM-based document rerankers via reinforcement learning. *arXiv preprint arXiv:2503.06034*.

Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. 2024b. A setwise approach for effective and highly efficient zero-shot ranking with large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 38–47.

A Prompt Templates

[Final prompt templates for MaxContext-DualEnd, MaxContext-TopDown, MaxContext-BottomUp, and the Top-m+Dualend cascade will be included here.]

B Hyperparameters and Implementation Notes

[Full hyperparameter tables for Phase 1–5 of the staged experiment matrix, including SLURM resource budgets and decoding settings, will be included here.]